MTD315

Examination - January Semester 2024

Computer Interactive Graphics

Monday, 15 April 2024 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **FOUR (4)** questions and comprises **TWELVE (12)** printed pages (including cover page).
2. You must answer **FOUR (4)** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

MTD315 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1a

Appraise WebGL for 3D use with **THREE (3)** Pros and **THREE (3)** Cons.

(12 marks)

Question 1b

Attributes, Varyings and Uniforms are three types of variables in shading language. Describe the purposes and functions of them.

(6 marks)

Question 1c

Illustrate and appraise Phong Model used in lighting and shading: write down the formula of Phong Model and explain the function of each parameter used in the formula.

(7 marks)

Question 2

Analyse the WebGL coding program (key parts ONLY) as shown below.

```
var webgl
var vsString = `
attribute vec3 position;
void main() {
    gl_Position = vec4(position, 1.0)
    gl_PointSize = 10.0; // Set the size of the points
}
`
var fsString = `
varying lowp vec4 vColor;
void main() {
    gl_FragColor = vColor;
}
`
```

```
function init(){
    //Init WebGL and Shaders
    initWebgl();
    initShader();

    let triangleVertexData = [
        0, 0.75, 0,
        -0.5, 0, 0,
        0.5, 0, 0,
    ]
    let PointvertexData = [
        0, 0.5, 0,
        0.1, 0.25, 0,
        -0.1, 0.25, 0,
    ];
    let colorDataA = [
        0.0, 1.0, 0.0, 1.0,
    ];
    let colorDataB = [
        1.0, 0.0, 0.0, 0.0,
    ]; //Set red color here

    //Create two buffers for triangle and points drawing
    bufferA = initbaseBuffer(triangleVertexData);
    webgl.clearColor(1.0,0.0,0.0,1.0);
    webgl.clear(webgl.COLOR_BUFFER_BIT);
    bufferB = initbaseBuffer(PointvertexData);

    //Sequentially draw elements based on different buffers
    draw_body(bufferA, colorDataA);
    draw_point(bufferB,PointvertexData,colorDataB);
}
```

```
86  //Declaring a function to init a buffer based on vertex data
87  function initbaseBuffer(vertexData){
88
89      let float = new Float32Array(vertexData);
90      const buffer = webgl.createBuffer();
91      webgl.bindBuffer(webgl.ARRAY_BUFFER, buffer);
92      //Comment#1 of Line93 goes here
93      webgl.bufferData(webgl.ARRAY_BUFFER, float, webgl.STATIC_DRAW);
94      return buffer;
95  }
96
97  //Declaring a function to draw elements based on buffer and color
98  function draw_body(buffer, color) {
99      //Comment#2 of Line100 goes here
100     webgl.bindBuffer(webgl.ARRAY_BUFFER, buffer);
101
102     //Comment#3 of Line103 goes here
103     const positionLocation = webgl.getAttribLocation(webgl.program, `position`);
104
105     //Comment#4 of Line106~107 goes here
106     webgl.enableVertexAttribArray(positionLocation);
107     webgl.vertexAttribPointer(positionLocation, 3, webgl.FLOAT, false, 0, 0);
108
109     //Comment#5 of Line110~111 goes here
110     var uColor = webgl.getUniformLocation(webgl.program, 'vColor');
111     webgl.uniform4fv(uColor, color);
112     //Comment#6 of Line113 goes here
113     webgl.drawArrays(webgl.TRIANGLES, 1, 3);
114 }
```

```
116 //Declaring a function to draw points based on buffer and color
117 function draw_point(buffer,points,color) {
118     //
119     webgl.bindBuffer(webgl.ARRAY_BUFFER, buffer);
120
121     //
122     const positionLocation = webgl.getAttribLocation(webgl.program, `position`);
123
124     //
125     webgl.enableVertexAttribArray(positionLocation);
126     webgl.vertexAttribPointer(positionLocation, 3, webgl.FLOAT, false, 0, 0);
127
128     //
129     var uColor = webgl.getUniformLocation(webgl.program, 'vColor');
130
131     //Comment#7 of for-loop below goes here
132     for (var i = 0; i < points.length; i += 3) {
133         //Comment#8 of Line134 goes here
134         webgl.uniform4fv(uColor, color);
135         //Comment#9 of Line136 goes here
136         webgl.drawDots(webgl.POINTS, i / 3, 1);
137     }
138
139 }
```

Question 2a

Explain the function of the coding blocks and write down the answers as comments in Line 92, 99, 102, 105, 109, 112, 131, 133 and 135. Remember to follow the guidelines originally on those lines.

(9 marks)

Question 2b

There are **FIVE (5)** errors/bugs hidden in the code, particularly **TWO (2)** in GLSL string and **THREE (3)** in JavaScript. Identify them and write down their respective solutions.

(10 marks)

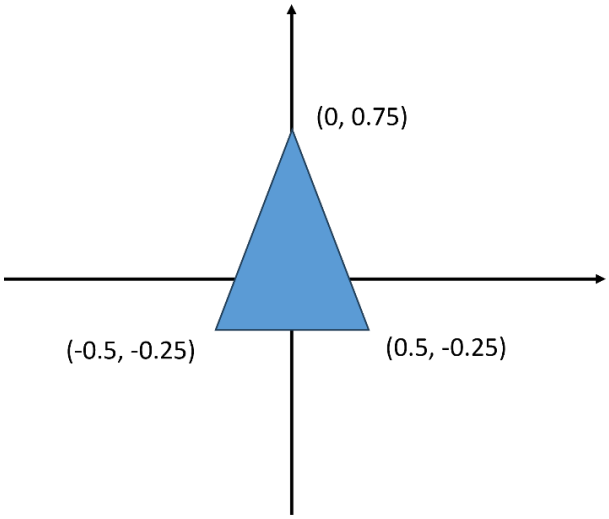
Question 2c

Draw a diagram **with coordinates** to represent the output of the corrected program.

(6 marks)

Question 3

A triangle sits at the origin in a 2D space as shown in the figure below.



Employ the use of affine transformations and corresponding methods from *glmatrix* to solve the following:

Question 3a

Find the combined transformation matrix M in the equation $\boldsymbol{v' = M*v}$ which translates the object 0.5 units along X-axis and then rotates the object 180 degrees counter-clockwise about Z-axis.

(3 marks)

Question 3b

Draw the triangle after transformation in Question 3a and indicate the coordinates of the new vertices.

(4 marks)

Question 3c

Complete the code blocks below by filling contents in Line 7 and 78 (i.e., *glmatrix* used in JavaScript and GLSL string) to realize the transformation in Question 3a using functions from *glmatrix*.

```
5   //Q3(c) answer goes here to complete GLSL for GPU vertex shader
6   var vsString = `
7
8   `
9   // GLSL for GPU fragment shader
10  var fsString = `
11  void main() {
12      gl_FragColor = vec4(1,1,0,1);
13  }
14  `
```

```
75      //Create Transformation Matrix 1
76      let modelView = mat4.create();
77      //Q3(c) answer goes here to create rotation matrix from modelView
78
79
80      // locate the transformation matrix on the GPU
81      let uniformMatrix = webgl.getUniformLocation(webgl.program, `u_forMatrix`);
82      webgl.uniformMatrix4fv(uniformMatrix, false, modelView);
83
84      //Create Transformation Matrix 2
85      let modelView2 = mat4.create();
86      mat4.identity(modelView2);
87      //create translation matrix
88      mat4.translate(modelView2, [0.5, 0, 0]);
89      // locate the transformation matrix on the GPU
90      let uniformMatrix2 = webgl.getUniformLocation(webgl.program, `t_forMatrix`);
91      webgl.uniformMatrix4fv(uniformMatrix2, false, modelView2);
```

(5 marks)

Question 3d

Find the combined transformation matrix M in the equation $\boldsymbol{v'} = \boldsymbol{M} * \boldsymbol{v}$ which rotates the object 180 degrees counter-clockwise and translates the object 0.5 units along X-axis then.

(4 marks)

Question 3e

Draw the triangle after transformation in Question 3d and indicate the coordinates of the new vertices.

(4 marks)

Question 3f

Assuming the JavaScript codes reused from Question 3c, complete the code blocks below (i.e., GLSL string) to realize the transformation in Question 3d using functions from *glmatrix*.

```
//Q3(f) answer goes here to complete GLSL for GPU vertex shader
var vsString = `

`
// GLSL for GPU fragment shader
var fsString = `
void main() {
    gl_FragColor = vec4(1,1,0,1);
}
`
```

(5 marks)

Question 4

Imagine that you were in a team who aims to develop a virtual car showroom. The targeted web page is shown as below.

The WebGL program framework has been provided as below and the 3D artist had already provided the 3D vertex data of the car in the WebGL program.

```
let vertexstring = `
attribute vec4 a_position;
uniform mat4 u_formMatrix;
attribute vec4 a_color;
varying vec4 color;
attribute vec2 a_outUV;
varying vec2 v_inUV;
void main(void){
    gl_Position = u_formMatrix * a_position;
    color = a_color;
    v_inUV = a_outUV;
}
`;
let fragmentstring = `
precision mediump float;
varying vec4 color;
uniform sampler2D texture;
varying vec2 v_inUV;
void main(void){
  gl_FragColor =texture2D(texture, v_inUV);
}
`;
```

```
var webgl;
var angle = 45;
var webglDiv
var uniformTexture;
var texture;
var moveState;
var mouseDownX ;
var mouseDownY;
var offsetX = 0;
var offsetY = 0;
var wheelMove = 0;
var rimStyle = 1;
function init() {
    initWebgl();
    initShader();
    initBuffer();
    initEvent()

}
```

```javascript
function initEvent() {
    document.onmousedown = mouseDown;
    document.onmousemove = mouseMove;
    document.onmouseup = mouseUp;
    document.onmousewheel = onmouseWheel;
    //Q4(b)i: To complete Button event handling here:
    document.getElementById('toggleButton1').addEventListener('click', function() {
        rimStyle = 1;
    });

}
function mouseDown(e) {
    moveState = true;

    mouseDownX = e.clientX;
    mouseDownY = e.clientY;
}
//Q4(a)i: Mouse move event handling goes here:
function mouseMove(e) {

}
```

```javascript
function mouseUp(e) {
    moveState = false;

}
function onmouseWheel(e){
    wheelMove += e.wheelDelta/100;
    initBuffer();

}
function initWebgl() {
    webglDiv = document.getElementById('myCanvas');
    webgl = webglDiv.getContext("webgl");
    webgl.viewport(0, 0, webglDiv.clientWidth, webglDiv.clientHeight);

}
function initShader() {…
}
```

```javascript
function initBuffer() {

    //   Car vertex data is hardcoded here
    let arr = [ ···
    ]

    let pointPosition = new Float32Array(arr);
    let aPsotion = webgl.getAttribLocation(webgl.program, "a_position");
    let triangleBuffer = webgl.createBuffer();
    webgl.bindBuffer(webgl.ARRAY_BUFFER, triangleBuffer);
    webgl.bufferData(webgl.ARRAY_BUFFER, pointPosition, webgl.STATIC_DRAW);
    webgl.enableVertexAttribArray(aPsotion);
    webgl.vertexAttribPointer(aPsotion, 4, webgl.FLOAT, false, 6 * 4, 0);

    let attribOutUV = webgl.getAttribLocation(webgl.program, "a_outUV");
    webgl.enableVertexAttribArray(attribOutUV);
    webgl.vertexAttribPointer(attribOutUV, 2, webgl.FLOAT, false, 6 * 4, 4 * 4);

   //Q4(a)ii: Transform Matrix with binding u_forMatrix Goes here


    //Q4(b)ii: Texture calling goes here with if-else statement


}
```

```javascript
function initTexture(imageFile) {
    let textureHandle = webgl.createTexture();
    textureHandle.image = new Image();
    textureHandle.image.src = imageFile;
    textureHandle.image.onload = function () {
        handleLoadedTexture(textureHandle)
    }
    return textureHandle;
}

function handleLoadedTexture(texture) {
    webgl.bindTexture(webgl.TEXTURE_2D, texture);
    webgl.pixelStorei(webgl.UNPACK_FLIP_Y_WEBGL, 666);

    webgl.texImage2D(webgl.TEXTURE_2D, 0, webgl.RGBA, webgl.RGBA, webgl.UNSIGNED_BYTE, texture.image);
    webgl.texParameteri(webgl.TEXTURE_2D, webgl.TEXTURE_MAG_FILTER, webgl.LINEAR);
    webgl.texParameteri(webgl.TEXTURE_2D, webgl.TEXTURE_MIN_FILTER, webgl.LINEAR);
    webgl.texParameteri(webgl.TEXTURE_2D, webgl.TEXTURE_WRAP_S, webgl.CLAMP_TO_EDGE);
    webgl.texParameteri(webgl.TEXTURE_2D, webgl.TEXTURE_WRAP_T, webgl.CLAMP_TO_EDGE);


    webgl.clearColor(0, 0, 0, 1);
    webgl.clear(webgl.COLOR_BUFFER_BIT | webgl.DEPTH_BUFFER_BIT);
    webgl.enable(webgl.DEPTH_TEST);
    webgl.activeTexture(webgl.TEXTURE0);
    webgl.bindTexture(webgl.TEXTURE_2D, texture);
    webgl.uniform1i(uniformTexture, 0);
    webgl.drawArrays(webgl.TRIANGLES, 0, 36);
}
```

Analyse the current WebGL program provided above and assemble elements of WebGL design to complete this WebGL program with the **TWO (2)** functions as per required below.

Question 4a

Function 1: Make the car rotate with interaction for 360° interactive viewing: When user presses the mouse and move, the car can rotate along. When user scrolls the mouse wheel, the car can be zoomed-in or out.

Question 4a(i)

Write the necessary code for Line 53 and below based on the comments left by the team lead.

(8 marks)

Question 4a(ii)

Write the necessary code for Line 144 and below based on the comments left by team lead.

(10 marks)

Question 4b

Function 2: User can select different rim style and the rim texture can be dynamically loaded and rendered accordingly.

Question 4b(i)

Write the necessary code for Line 63 and below based on the comments left by team lead.

(3 marks)

Question 4b(ii)

Write the necessary code for Line 147 and below based on the comments left by the team lead.

(4 marks)

----- END OF PAPER -----